

SIGHTLINE

Design & Implementation Specification

VTHacks 14 • Working Title • Team / Devin Handoff

ONE-LINE PRODUCT: An AI operating system for the physical world — Ray-Ban Meta glasses give AI a first-person view, the phone adds audio + location, and verified agents remember, reason, and act.

Status	Hackathon MVP design
Primary demo	Live Ray-Ban POV → object memory → contextual alert → recall → verified Guardian action
Primary track targets	GoDaddy / ANS, Gemini, ElevenLabs, Tiger Data; Presage optional
Date	September 19, 2026

Design principle: Build one magical loop that works live. Everything else is an extension.

1. Executive Summary

SIGHTLINE turns Ray-Ban Meta glasses into a first-person sensor for an agentic AI system. The glasses provide the wearable camera and headset experience; an iPhone acts as the gateway for Meta's Wearables Device Access Toolkit (DAT), GPS, audio routing, permissions, and streaming; a laptop hosts the "Mission Control" UI where judges can see the wearer's live point of view, detected objects, events, locations, memories, missions, and agent activity.

The MVP is intentionally narrow: prove a reliable real-world loop where SIGHTLINE sees an object, remembers an event, notices a context change, alerts the wearer, and later answers a memory query. A second "Guardian" flow shows how the same infrastructure can recognize a possible emergency event and prepare a verified response workflow without claiming to diagnose medical conditions or autonomously call emergency services.

MVP SUCCESS = The judge watches the wearer's Ray-Ban POV live on the laptop, sees a backpack become a tracked object, watches SIGHTLINE notice the wearer leave it behind, hears the glasses warn the wearer, and then sees SIGHTLINE retrieve the last-seen frame + timestamp + location on command.

Product thesis

- AI understands the digital world extremely well; SIGHTLINE gives it a structured, real-time understanding of the physical world.
- The product is not "smart glasses AI." The glasses are the sensor; the product is the memory + mission + verified-action layer.
- The same observation stream should support many missions: remember objects, protect items, reconstruct events, locate safety resources, assist accessibility, or coordinate a search.
- Because the system can access first-person vision, location, and actions, every high-trust agent should have a verifiable identity. GoDaddy ANS becomes a security primitive, not a decorative sponsor integration.

2. MVP Scope and Non-Goals

P0 — Must work for judging

Capability	Acceptance criterion
Live Ray-Ban feed	Ray-Ban camera → iPhone DAT → laptop Mission Control with low enough latency for a convincing live demo.
Object detection	Display basic bounding boxes / labels for common objects (backpack, laptop, bottle, person, etc.).
Memory events	Persist "object seen / object placed / object last seen" with time, frame reference, and phone location.
Mission: Track item	User says or taps "track my backpack"; system maintains a lightweight tracked-object state.
Contextual alert	When wearer moves away and target remains behind / disappears for a threshold, speak an alert.
Recall query	"Where did I leave my backpack?" returns last-seen time, image, and location/context.

Capability	Acceptance criterion
Voice output	ElevenLabs response is delivered through the phone / glasses audio route.
ANS trust gate	At least one agent is registered/resolved/verified through ANS; dashboard visibly allows verified requests and blocks a simulated unverified agent.

P1 – Add only after P0 is reliable

- Guardian simulation: detect an intentional fall / “help” cue, create a POSSIBLE EMERGENCY event, capture location, and open a simulated escalation panel.
- Presage overlay on a relatively stationary consenting teammate to show camera-derived wellness signals. Treat this as informational only, never as diagnosis.
- Accessibility mission: identify exits, stairs, obstacles, signs, or other user-requested environmental cues.
- ATLAS safety-resource memory: remember previously seen AED / exit / first-aid signage and retrieve the most recent observation.

P2 – Stretch / pitch as roadmap

- Search Party: multiple opt-in phones / glasses contribute sightings to a temporary mission.
- Marketplace mission: “What is this worth?” / “Sell this” generates a draft listing after item identification and verified agent handoff.
- Multi-agent dynamic mission creation: spawn different agent roles based on natural-language mission intent.
- Indoor routing / full 3D spatial mapping / AR arrows.

NON-GOALS FOR THE HACKATHON: facial recognition; medical diagnosis; autonomous 911 dialing; permanent 24/7 surveillance; full SLAM; unsupervised financial transactions; dozens of fragile sponsor integrations.

3. Four-Minute Judge Demo

Four-Minute Live Demo — Judge Experience



Optional only if stable: Presage wellness overlay • Search Party multi-device mission • “Sell this” marketplace agent

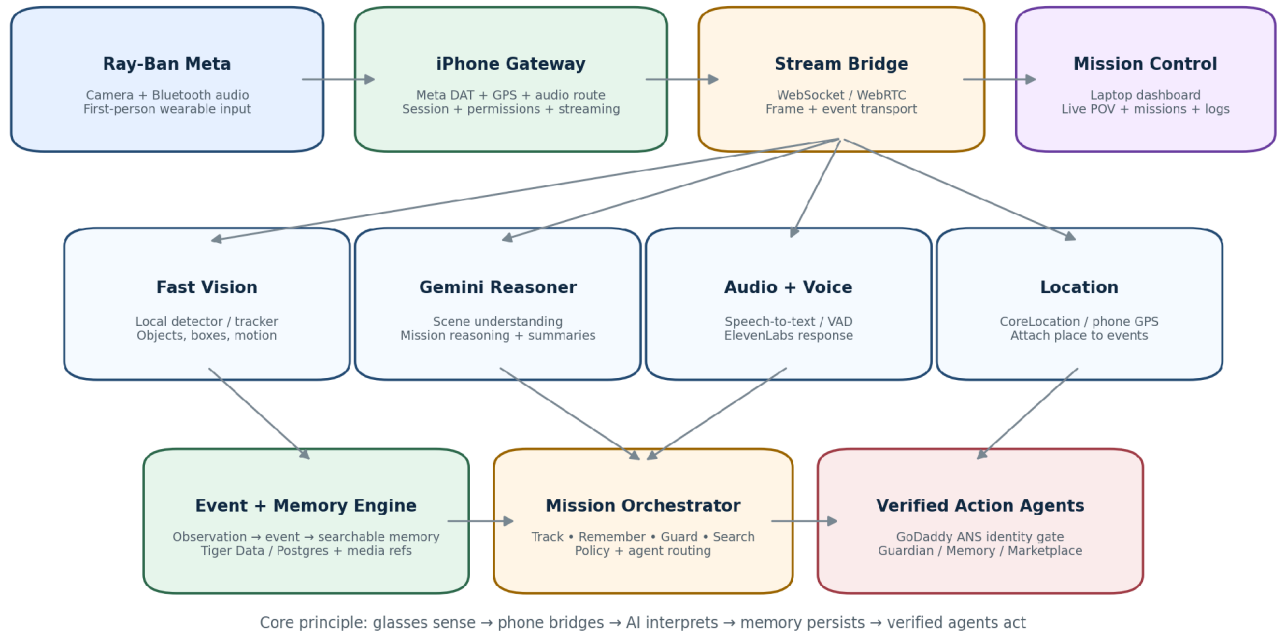
Time	Beat	What the judge sees / hears
0:00–0:25	Problem + thesis	“AI can search the internet, but it has almost no memory of the physical world around us. SIGHTLINE gives verified AI agents eyes, ears, location, and memory.”
0:25–0:50	Prove the hardware loop	Show the live Ray-Ban first-person stream on the laptop. Look at 3–4 objects and let boxes / labels appear.
0:50–1:30	Track mission	Say: “Sightline, track my backpack.” Put it down. The dashboard creates a mission and stores the observation.
1:30–1:55	Contextual alert	Walk away. SIGHTLINE speaks: “Your backpack is still behind you.” Judge sees the trigger and event timeline.
1:55–2:25	Memory recall	Ask: “Where did I leave my backpack?” Show last-seen frame, time, and current/last-known location context.
2:25–3:05	Guardian simulation	Teammate performs a safe scripted fall / says “help.” Dashboard creates POSSIBLE EMERGENCY. Show location capture and simulated response plan.
3:05–3:35	ANS trust demo	Guardian agent requests sensitive location/event data → ANS verified → allowed. Simulated unknown agent → blocked.
3:35–4:00	Close	Show architecture slide briefly. “One sensor stream, many missions. Today we built memory and safety; tomorrow it can power search, accessibility, field operations, and commerce.”

Demo reliability rules

- Have a clearly labeled “DEMO MODE” for Guardian so no real emergency call is triggered.
- Use a backpack / laptop with a visually distinct appearance. Do not make the live demo depend on tiny objects like AirPods.
- Pre-test the exact path and lighting where judging will happen.
- If Ray-Ban connectivity fails, switch to Meta’s mock / phone-camera streaming fallback while preserving the same software stack and explain the fallback transparently.
- Do not make Presage or multi-device Search Party part of the only working path. They are bonus moments.

4. System Architecture

SIGHTLINE — Proposed System Architecture



4.1 Wearable layer — Ray-Ban Meta

- Use Meta Wearables Device Access Toolkit (DAT) from a native iOS app. Current Meta documentation explicitly supports camera/video streaming from Ray-Ban Meta into iOS/Android mobile applications.
- Use the glasses as a first-person camera and open-ear audio headset. The phone remains the actual application host and network bridge.
- Enable Developer Mode in the Meta AI app and validate camera streaming on-device before any higher-level AI work.

4.2 iPhone gateway

- Swift app with Meta DAT SDK packages for registration/session/camera streaming.
- CoreLocation attaches coarse or precise location (with permission) to significant events rather than every frame.
- Audio route uses iOS Bluetooth microphone/speaker profiles. Voice activity detection and speech-to-text can run on phone or backend.
- The gateway should package frames/events into a simple transport protocol and send them to the backend/laptop over Wi-Fi.
- Keep a local reconnect loop and surface session state: CONNECTED / STREAMING / DEGRADED / DISCONNECTED.

4.3 Streaming strategy

Do not send every camera frame to a large multimodal model. Split the stream into a fast path and a smart path:

Ray-Ban stream

- └─▶ Preview path → laptop UI (near real time)
- └─▶ Fast vision path → local detector / tracker (multiple frames per second)
- └─▶ Smart reasoning path → sampled frames + recent transcript + mission state → Gemini

- Preview path makes the project feel live.
- Fast vision path handles labels, bounding boxes, object continuity, and simple motion state cheaply.
- Gemini handles scene semantics, ambiguous intent, event descriptions, and cross-modal reasoning. Sample frames when state changes or at a low rate instead of every frame.

5. Core Software Modules

Module	Suggested stack	Responsibility
Wearable Gateway	iOS / Swift	DAT session, camera frames, Bluetooth audio route, GPS, permission state, reconnect logic.
Stream Bridge	Node.js or Python	WebSocket/WebRTC transport, frame fan-out, event ingestion, session IDs.
Vision Service	Python	Object detector + tracker; lightweight “target still present?” logic.
Gemini Reasoner	Server	Scene descriptions, event extraction, mission intent, safety-event reasoning, summaries.
Memory Engine	Tiger Data / Postgres	Time-series observations, object sightings, locations, mission events, search queries.
Mission Orchestrator	Server	Maintains active missions and evaluates triggers / policies.
ANS Trust Layer	Server	Resolve / verify agent identity before sensitive data or action handoff.
Voice Service	ElevenLabs	Natural spoken responses / alerts routed back to wearer.
Mission Control UI	Next.js / React	Live POV, boxes, timeline, map/location context, missions, agent status, ANS verification.

6. Mission Model

A “mission” is the product abstraction. A mission tells the system what to care about, which observations matter, what trigger conditions to watch, and what actions are allowed. This keeps SIGHTLINE from becoming generic surveillance and gives judges a clean mental model.

6.1 Track / Watchtower mission (MVP)

User intent: “Don’t let me lose my backpack.”
 Target: backpack track ID + visual descriptor
 State: observed → placed → wearer moved away → target absent
 Trigger: distance/time threshold crossed
 Action: spoken alert + save last-seen event

6.2 Memory / Anchor mission (MVP)

User intent: "Remember where I leave this."
Capture: target image + label + timestamp + phone location
Update: last_seen_at whenever target is confidently re-observed
Query: "Where did I leave my backpack?"
Response: last-seen frame + time + location/context + spoken summary

6.3 Guardian mission (P1)

Signals (example): sudden posture change + person on ground + "help" audio cue
System output: POSSIBLE EMERGENCY event (not diagnosis)
Actions: freeze relevant event context → capture location → search remembered safety resources → prepare simulated escalation
High-trust data/action access: gated through verified Guardian agent identity

6.4 Search Party mission (P2)

- Create a temporary mission with a non-biometric target description: clothing, bag, object, or explicitly opted-in marker.
- Multiple opt-in devices post sightings with time/location/confidence.
- Server deduplicates sightings and displays a movement trail. Avoid uncontrolled face recognition in the hackathon.

6.5 Marketplace mission (P2)

- "What is this worth?" → identify object → search current market comparables → estimate range.
- "Sell this" → verified marketplace/listing agent requests item photo + description → ANS verification gate → draft listing. Do not auto-publish during demo unless integration is reliable and authorized.

7. Data Model / Event Contracts

Keep the schema intentionally simple. The AI layer can be sophisticated without a complicated database.

```
Observation {
  id, session_id, timestamp, frame_ref,
  source: "rayban" | "phone" | "mock",
  location?: {lat, lng, accuracy_m},
  transcript_window?: string,
  detections: [{track_id, label, confidence, bbox}]
}

Event {
  id, type, timestamp, mission_id?, severity,
  subject_track_id?, description, frame_ref?, location?,
  model_confidence?, verified_by_user?: boolean
}

Mission {
  id, type, status, target_descriptor,
  trigger_config, allowed_actions, created_at
}
```

```
AgentAccessRequest {
  agent_ans_name, requested_scopes[], mission_id,
  verification_status, decision, timestamp
}
```

Event type	Meaning	Example
object_seen	A recognized target is visible.	backpack track 42 visible at sponsor table
object_last_seen	Target disappears or wearer leaves context.	last seen 12:54:31 near table 17
commitment_heard	Optional voice/context extraction.	"send the deck tonight"
possible_emergency	Safety heuristics crossed; needs human confirmation / simulation.	person fell + help cue
agent_verified	ANS identity check passed.	guardian.<domain> ACTIVE
agent_blocked	Identity could not be trusted.	unknown agent requested location

8. GoDaddy ANS Integration — Primary Sponsor Story

ANS should protect the exact thing that makes SIGHTLINE powerful: access to first-person sensor context and the ability to take real-world actions. The architecture should treat an agent identity check as a visible gate before an agent receives sensitive event/location data or performs a high-trust action.

Recommended agent identities

```
ans://v1.0.0.observer.<team-domain>
ans://v1.0.0.memory.<team-domain>
ans://v1.0.0.guardian.<team-domain>
ans://v1.0.0.action.<team-domain>
ans://v1.0.0.market.<team-domain> # optional
```

ANS demo flow

1. Guardian mission produces a POSSIBLE EMERGENCY event.
2. Guardian agent requests current location + recent event context.
3. Backend resolves/verifies the agent through ANS and checks that its identity/version is active.
4. Mission Control animates "VERIFYING AGENT..." → "VERIFIED". Data is released.
5. Repeat with a simulated unknown/unregistered agent. Mission Control shows "BLOCKED — unverified agent".

Pitch line: "If AI is going to see through your eyes and act in the physical world, you need to know exactly which agent you are trusting. SIGHTLINE uses ANS to verify agent identity before sensitive context is released."

9. Sponsor / MLH Track Strategy

Track	Priority	How it fits
GoDaddy / ANS	PRIMARY	Agent identity and trust gate for vision/location/action access. Make verification visible in the demo.
Gemini API	PRIMARY	Multimodal scene reasoning, mission intent, event descriptions, memory-query reasoning.
ElevenLabs	PRIMARY	Hands-free voice personality and urgent/contextual spoken alerts through the Ray-Ban audio route.
Tiger Data	PRIMARY / strong fit	High-frequency event/time-series store; last-seen queries; event timelines; real-time dashboard data.
Presage	OPTIONAL	Wellness signals from a consenting, sufficiently stationary subject. Informational only; do not make it the emergency detector.
Vultr	OPTIONAL	Host backend / inference if cloud GPU or quick deployment materially helps.
MongoDB Atlas	OPTIONAL / choose instead of Tiger	Could store semantic memory and vector search, but avoid running two databases only to collect tracks.
GoDaddy Registry	EASY ADD	Use a memorable project domain for the submission and live dashboard.
Best UI/UX	STRONG	Mission Control is inherently visual: live POV, detections, timeline, missions, agent trust state.
Best Ut Prosim	STRONG	Guardian / Search Party / safety-resource memory directly serve people and community response.
Best DEI	OPTIONAL, meaningful only	Accessibility mission for requested environmental cues; build only if it is genuinely useful and testable.
Solana	SKIP unless needed	Could anchor provenance hashes, but it is not necessary to the product and would dilute the story.

Other sponsor tracks such as Capital One and Impiricus currently have public Devpost entries with challenge details still listed as TBD. Do not distort the product around them until the actual challenge statement is known. If their live challenge aligns, use SIGHTLINE as a platform extension rather than changing the core demo.

10. Privacy, Safety, and Responsible Demo Boundaries

- Visible consent: only analyze teammates / judges in ways they have explicitly agreed to during the demo.
- No uncontrolled face recognition. Identify target objects or temporary non-biometric visual descriptors instead.
- Store key event frames, not necessarily continuous raw video. Use short retention for the hackathon.
- Guardian says “possible emergency” and asks for confirmation / operates in simulation mode. It does not diagnose a heart attack, stroke, seizure, or other medical condition.

- Do not actually call emergency services during judging. A production emergency feature would require separate safety engineering, regulatory review, and false-positive controls.
- Presage signals should be described as camera-derived wellness/physiological estimates within the provider's intended-use limitations, not clinical diagnosis.
- ANS verification proves agent identity/trust metadata; application authorization policies still decide what that verified agent may access.

11. Implementation Plan for Devin / Team

ORDER MATTERS: Do not build Guardian, Presage, Search Party, or marketplace actions until the glasses → phone → laptop live stream is stable.

Phase	Goal	Definition of done
Phase 0	Repository + contracts	Create /ios-gateway, /backend, /dashboard. Define shared event JSON. Add .env.example and README quick start.
Phase 1	Ray-Ban → iPhone	Integrate Meta DAT iOS sample patterns. Show local camera preview. Handle permission/session lifecycle. Confirm physical glasses stream.
Phase 2	iPhone → laptop	Send frames over WebSocket (or WebRTC if already easy). Display live preview in dashboard. Add reconnect/status UI.
Phase 3	Fast vision	Run detector/tracker on server/laptop. Overlay labels/boxes in Mission Control. Emit object_seen observations.
Phase 4	Memory mission	Persist sightings; add "Track this" mission; determine last-seen event; implement recall query and frame card.
Phase 5	Voice + location	Attach CoreLocation to events; capture push-to-talk or VAD speech; ElevenLabs replies; route playback through glasses if stable.
Phase 6	ANS	Register/resolve at least one agent. Build verifyAgent() middleware and dashboard verification animation. Add fake unverified-agent test.
Phase 7	Guardian simulation	Add simple multimodal trigger rules + Gemini reasoning; produce POSSIBLE EMERGENCY event; show simulated escalation panel.
Phase 8	Polish	Demo reset button, seeded fallback data, loading states, confidence display, error states, fullscreen Mission Control, practice 4-minute script.
Stretch	Only if time remains	Presage overlay, ATLAS/AED memory, Search Party, marketplace draft flow.

12. Devin Build Instructions (Paste as Project Brief)

You are building SIGHTLINE, a VTHacks 14 hackathon project.

CORE PRODUCT:

SIGHTLINE is an AI operating system for the physical world. Ray-Ban Meta glasses provide a first-person camera; an iPhone is the Meta DAT + GPS + audio gateway; a laptop web dashboard shows the live POV, object detections, event timeline, missions, memory, and verified agent actions.

ABSOLUTE PRIORITY:

First make Ray-Ban Meta live camera streaming work from the official Meta Wearables Device Access Toolkit in a native iOS app. Then forward that stream to the laptop dashboard. Do not build advanced features until this path is stable.

REPO SHAPE:

/ios-gateway Swift iOS app using Meta DAT
/backend Node/Python service for frames, events, mission state, AI calls
/dashboard Next.js/React Mission Control
/shared JSON schemas/types if useful

MVP FLOW:

1. Wear Ray-Bans; laptop displays live POV.
2. Detect common objects and draw boxes/labels.
3. User starts a "track my backpack" mission.
4. Persist sightings with timestamp + frame reference + phone location.
5. When wearer walks away and the backpack remains behind/disappears, create an alert event and speak a warning.
6. "Where did I leave my backpack?" returns last-seen image/time/location.
7. Implement an ANS identity gate for a Guardian/Action agent. Show verified agent = allowed, unverified simulated agent = blocked.
8. If time remains, add a Guardian simulation for a scripted fall/help event.

TECHNICAL GUIDELINES:

- Use official Meta DAT iOS docs/sample code. Respect session/permission lifecycle.
- Phone is the bridge; do not attempt direct glasses-to-laptop pairing.
- Use WebSockets for the first working stream unless WebRTC is already trivial.
- Separate fast object detection/tracking from slower Gemini scene reasoning. Do not send every video frame to Gemini.
- Use Gemini for multimodal scene/event reasoning and natural-language mission intent.
- Use ElevenLabs for spoken alerts/responses.
- Use phone CoreLocation for event location.
- Prefer one coherent primary database (Tiger Data/Postgres is preferred) over adding multiple databases for prizes.
- Agent identity verification through GoDaddy ANS must be visible in the UI.
- No facial recognition, medical diagnosis, autonomous emergency calls, or unsafe demo behavior.

DASHBOARD:

Dark, polished Mission Control UI. Large live POV at left. Right rail: detections, current location, transcript, active missions, agent status. Bottom: chronological event timeline. Show connection state and one-click Demo Reset.

FAILURE HANDLING:

If physical glasses fail, support Meta mock/phone-camera input using the same pipeline. Never make the UI crash because one AI provider fails; degrade gracefully and continue preview/memory.

DO NOT OVERBUILD. The live POV + memory + alert + recall loop is the product proof.

13. Mission Control UI Specification

Area	Behavior / content
Header	SIGHTLINE logo/name; LIVE dot; Ray-Ban connection status; phone battery/network if easy; active mission count; Demo Reset.
Left 65%	Live first-person video. Bounding boxes and small labels. Avoid clutter. Selected target gets a persistent accent outline.
Right rail — Vision	Top detections + confidence; tracked target; last seen time.
Right rail — Context	Current location label; current transcript snippet; system mode.
Right rail — Agents	Observer, Memory, Reasoner, Guardian, Action. Show IDENTITY VERIFIED / STANDBY / BLOCKED states.
Bottom timeline	Event chips/cards ordered newest-first: SEEN, PLACED, LEFT BEHIND, ALERTED, RECALLED, POSSIBLE EMERGENCY, AGENT VERIFIED.
Memory card	Last-seen frame thumbnail + object label + timestamp + location + “why we think this” short explanation.
Guardian panel	Big amber/red panel only in demo simulation: signals observed, confidence, location captured, next suggested actions. Must say SIMULATION MODE.
ANS modal	Show ans:// identity, verification status, trust/identity information if available, and access scope being requested.

14. Acceptance Tests

Test	Pass condition
Glasses connection	Can connect, start stream, stop stream, recover from brief disconnect without restarting everything.
Latency	Judge can move their head / object and see a convincingly live response on laptop.
Object track	Backpack maintains stable identity long enough for the scripted demo.
Memory	After moving away, last-seen query returns the correct event and image.
Location	Location is attached to significant events when permission is granted; app degrades gracefully if not.
Voice	At least one spoken command and one ElevenLabs spoken response work hands-free or with a documented push-to-talk fallback.
ANS	Verified agent path succeeds; unverified test agent is denied and visibly explained.
Guardian	Scripted simulation creates possible_emergency without making clinical claims or real emergency calls.
Reset	One button clears demo session state and prepares for the next judge.

Test	Pass condition
Offline/failure	If Gemini/ANS/DB call fails, live preview continues and UI shows a clear degraded state.

15. Risks and Fallbacks

Risk	Level	Mitigation
DAT developer-preview instability	HIGH	Test physical glasses first. Keep phone-camera / MockDevice fallback. Surface reconnect state.
Wi-Fi latency / congestion	HIGH	Run backend locally on laptop when possible; use local LAN; compress frames; sample AI frames separately.
Object tracking instability	MED	Use a large distinct backpack; allow manual target lock; combine detector label + appearance crop.
Audio routing complexity	MED	Ship with push-to-talk from phone/laptop if full glasses mic pipeline is unstable. Keep ElevenLabs output.
Gemini latency/cost	MED	Trigger on state changes, not every frame. Cache descriptions. Keep fast detector independent.
Presage motion limitations	MED	Treat as optional; demo on stationary consenting teammate if used.
False emergency event	MED	Require multiple signals and/or explicit confirmation; always show SIMULATION MODE during judging.
Scope explosion	HIGH	Freeze P0 after it works. Do not add sponsor tracks unless they strengthen the same demo story.

16. Pitch Narrative

OPEN: “AI can search nearly everything we have ever put online. But it still forgets what happened five feet in front of us five minutes ago.”

Then immediately show the live first-person stream. Do not spend the first minute explaining architecture. Let the judge experience the magic first.

CLOSE: “SIGHTLINE gives AI eyes, ears, location, and memory — then uses verified agents to act safely on that understanding of the physical world.”

17. Verified Technical References

Source	Why it matters	URL
Meta Wearables FAQ	Confirms DAT extends iOS/Android apps with AI-glasses video streaming, photo capture, microphone/audio capabilities, supported Ray-Ban Meta devices, and local/cloud processing.	https://developers.meta.com/wearables/faq/
Meta Wearables DAT iOS repository	Official iOS SDK repository; includes camera-streaming sample, session/permission guidance, MockDevice testing, and setup.	https://github.com/facebook/meta-wearables-dat-ios
Meta DAT CameraAccess sample	Official sample explicitly demonstrates connecting to wearable devices and streaming live video from Ray-Ban Meta cameras.	https://github.com/facebook/meta-wearables-dat-ios/blob/main/samples/CameraAccess/CameraAccess/CameraAccessApp.swift
GoDaddy ANS developer documentation	ANS registration/resolution/verification and agent identity naming; supports agent registration and domain validation.	https://www.godaddy.com/ans/developers
GoDaddy ANS verification article	Explains offline agent identity verification and separation of identity, proof-of-possession, and authorization.	https://www.godaddy.com/resources/news/dont-trust-verify-offline-sub-millisecond-agent-verification-with-ans
VTHacks 14 Devpost	Current public prize categories, MLH tracks, judging criteria, deadline, and four-minute presentation requirement.	https://vthacks-14.devpost.com/

End of design specification